Jaypee University of Information Technology, Waknaghat Solan H.P

Solution Sheet
Shortest Path Algorithms

Q1) Answer: a
For a **directed acyclic graph (DAG)**, the fastest known algorithm for single-source shortest paths uses the following two steps: (1) compute a topological ordering of the vertices in $\Theta(m+n)$ time, then (2) process vertices in that topological order, relaxing every outgoing edge of each vertex exactly once, which takes $\Theta(m+n)$ time in total (this correctly computes shortest paths since, in topological order, all edges into a vertex are relaxed before that vertex is finalized, and no negative cycles can exist in a DAG so ordinary Dijkstra-style priority queues or Bellman-Ford repeated relaxation are unnecessary).

This is strictly faster than Dijkstra's $\Theta(m+n\log n)$ (needed only for general graphs with a priority queue) or the $\Theta(nm)$Bellman-Ford bound (needed for general graphs, possibly with negative edges but no negative cycles) — the DAG structure lets us avoid both.

Q2) Answer: a
$d_k(u)$ is the shortest-path weight from s to u using **at most k edges** — this is exactly the quantity computed by the Bellman-Ford relaxation process after k rounds.

S1: $d_{k+1}(u) \le d_k(u)$— allowing one extra edge of "budget" can only give access to more paths (a path of length $\le k$is also a path of length $\le k+1$), so the shortest weight can only stay the same or decrease. **True**, always.

S2: Suppose (u,v) lies on a shortest path from s to v; then $d(v) = d(u) + w(u,v) \ge d(u)$in the limit, so eventually $d_k(u) \le d_k(v)$once ***k* is large enough**. But for small k this need not hold: it's possible that u has not yet been reached with a short enough path (so $d_k(u) = \infty$ is even bigger) or, more subtly, that a different, cheaper path of length $\le k$ to v exists that doesn't go through u at all — the inequality can fail for some particular k even though it eventually holds in the limit. So S2 is **not always true for every** k.

Q3) Answer: c
Adding the same positive constant α to **every** edge shifts the total weight of *any* spanning tree by exactly $\alpha \times (n-1)$(since every spanning tree has exactly $n-1$ edges, the same count) — a uniform shift that doesn't change which spanning tree has the minimum total weight. So **every MST remains an MST**.

Shortest *paths*, however, can use a *different number of edges* depending on the route — a path with fewer edges gets a smaller total addition of α than a path with more edges, which can flip which path is actually shortest after the update. So **SPs need not remain SPs**.

Q4) Answer: c
Let the **diameter** of the graph be the maximum shortest distance between any two vertices.
It is given that $\max_{u,v\in V, u\neq v} d(u,v) = 30$.
Thus, the **diameter** of the graph is 30.

Relation between BFS Tree Height and Graph Diameter

A Breadth-First Search (BFS) tree rooted at a vertex (r) has the following property:

- The level of every vertex equals its shortest distance from the root.
- Therefore, the height of the BFS tree is the maximum shortest distance from the root to any vertex, which is called the **eccentricity** of the root.

Let the height of the BFS tree be (h). Then, $h = \max_{v\in V} d(r,v)$.

Using the Diameter

Let the diameter of the graph be (D=30). A well-known property of graphs is Radius $\geq \left\lceil \frac{D}{2} \right\rceil$.

Since $D = 30$,we get $\left\lceil \frac{30}{2} \right\rceil = 15$.

Every vertex has eccentricity at least the radius. Hence, regardless of the chosen BFS root, $h \geq 15$.

The height need not be exactly 15 or exactly 30. It can be any value between 15 and 30 depending on the chosen root.

For example:

- If the BFS starts from a central vertex, the height can be 15.
- If it starts from an end vertex of a diameter, the height can be 30.

Q5) Answer: a
Checking each statement:
- (I): distinct weights ⇒unique MST True
- (II): two different paths can have equal total weight even with distinct edges (e.g. 1 + 4 = 2 + 3) False

Q6) Answer: 12
Without edge {3, 4}, the shortest path from 3 to 4 is 3-2-1-4 = 5 + 2 + 5 = 12.

The edge of weight x lies on a shortest path (at least one, possibly tied) only if $x \leq 12$. The largest integer is 12.

Q7) Answer: c
Bellman-Ford takes $\Theta(V\,E)$. A complete graph has $E = \Theta(n^2)$.

$$n \times n^2 = \Theta(n^3)$$

Q8) Answer: SACET
Dijkstra from S: B = 3, A = 4, D = 7 (via S).
- Pop B: D via B = 7, not strictly shorter, so D keeps parent S.
- Pop A: C = 5. Pop C: E = 6 (D via C = 8, no change).
- Pop E: T = 10 (via E), G = 8.
- Pop D: T via D = 10, not strictly shorter, so T keeps parent E.

Path: S →A →C →E →T

Q9) Since all edges have weight 1, we use BFS (Breadth First Search) to find the shortest path.
We mark a node visited when it is inserted into the queue.

Step	Dequeued	Newly visited (enqueued)	Queue after operation
0	—	0	[0]
1	0	1, 3	[1, 3]
2	1	2	[3, 2]
3	3	4	[2, 4]
4	2	6	[4, 6]
5	4	5	[6, 5]
6	6	7, 8	[5, 7, 8]
7	5	None	[7, 8]
8	7	None	[8]
9	8	None	[]

Construct shortest path using parent array
Whenever we insert a new vertex, record its parent.

Vertex	Parent
0	NIL
1	0
3	0
2	1
4	3
6	2
5	4
7	6
8	6

Now trace backwards from 8:
8<- 6<- 2<- 1<- 0
Therefore path:
0 -> 1 -> 2 -> 6 -> 8
Shortest distance = 4 edges.

Q10) Using Bellman-Ford algorithm

1. Initialize source distance to 0 and all other distances to infinity.
2. Relax every edge repeatedly for V-1 iterations.
3. Check all edges one additional time.
4. If any distance can still be reduced, a reachable negative-weight cycle exists.

The minimum-cost path from a to f is:

a-> d-> b-> c-> e-> f

Q11) Using Kahn's algorithm

1. **Find the Topological Order:** Run Kahn's algorithm to process the DAG and generate a valid topological ordering of the vertices.
2. **Relax Edges in Order:** Initialize the source vertex distance to 0 and all other vertices to infinity (∞). Traverse the vertices in the topological order obtained from Kahn's algorithm and relax the outgoing edges for each reachable vertex.

There are two shortest paths:

Path 1: A → B → D → F

Path 2: A → B → E → F

Q12) After applying Floyd Warshall Algorithm final minimum distance between each pair is.

	0	1	2	3	4
0	0	4	5	5	7
1	3	0	1	4	6
2	2	6	0	3	5
3	3	7	1	0	2
4	1	5	5	4	0

Each cell dist[i][i] in the output shows the shortest distance from node i to node j, computed by considering all possible intermediate nodes using the Floyd-Warshall algorithm.

Q13) Two steps of Johnson's Algorithm:

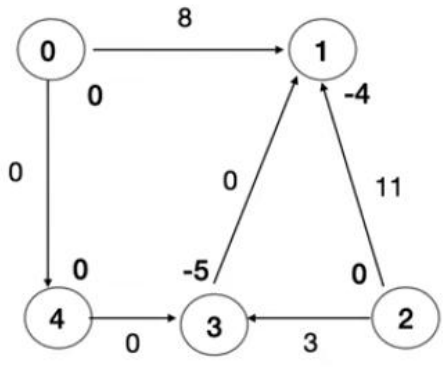
1. Reweight the edges: Add a new vertex s with zero-weight edges to all vertices. Run Bellman-Ford to calculate vertex potentials h(v), then reweight each edge:

$w'(u,v)=w(u,v)+h(u)-h(v)$

2. Run Dijkstra's Algorithm: Run Dijkstra from every vertex using the reweighted edges to find shortest paths between all pairs of vertices. Convert distances back to original weights.

$d(u,v)=d'(u,v)-h(u)+h(v)$

After applying first step we get updated weights of edges as follows:



w(u, v)'= d[u] + w(u, v) - d[v]

Q14) Answer: a

For any path $s = v_0 \to v_1 \to \cdots \to v_k = t$, the f-terms cancel in pairs:

$$w'(\text{path}) = w(\text{path}) + f(s) - f(t)$$

Every s-to-t path changes by the same constant $f(s) - f(t)$, so the shortest one stays shortest, for **any** f. (This is the reweighting trick in Johnson's algorithm.)

Q15)

Node (x,y)	Heuristic value (h(n)) Floor (Euclidian distance between Goal G (14,9) and given node.)
1) (4,5)	**10**
2) (7,9)	**7**
3) (7,6)	**7**
4) (9,4)	**7**
5) (11,8)	**3**
6) (14,9)	**0**

A* algorithm:
At each step, A* calculates a total cost value f(n) for every neighbouring node n using the following formula:

f(n)=g(n)+h(n)

- **g(n):** The **exact cost** of the path from the starting node to the current node n.
- **h(n):** The **heuristic estimate** of the cost to get from node n to the final goal.
- **f(n):** The **total estimated cost** of the cheapest path through node n.

The algorithm always prioritizes expanding the node that has the **lowest f(n) value**.

After implementing the algorithm minimum path comes out to be:

S(4,5) => (7,6) => (11,8) => G(14,9)